ESTIMATE MEDICAL INSURANCE COST USING LINEAR REGRESSION

A Major Project Report Submitted to

KAMARAJ UNIVERSITY

in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF SCIENCE COMPUTER SCIENCE

Submitted by

MADHU.V (Reg. No: C4S22373)

Under the Guidance of

S.YUVAKARTHIKA, M.SC.,ASSISTANT

PROFESSOR ,DINDIGUL

COMPUTER SCIENCE

NPR ARTS AND SCIENCE COLLEGE

KAMARAJ UNIVERSITY

2024-2027



NPR ARTS AND SCIENCE COLLEGE

COMPUTER SCIENCE

BONAFIDE CERTIFICATE

Certified that this project report titled "ESTIMATE MEDICAL INSURANCE COST USING LINEAR REGRESSION" is the bonafide work of V. MADHU (Register No: C4S22373) who carried out the project work under my supervision during the academic year [2024-2027], in partial fulfilment of the requirements for the award of the degree of BSC.COMPUTER SCIENCE of KAMARAJ UNIVERSITY.

Internal Guide Head of the Department

Submitted for the Viva-Voce Examination held on _____ at NPR ARTS AND SCIENCE COLLEGE

Internal Examiner External Examiner

DECLARATION

I hereby declare that this project work entitled "ESTIMATE MEDICAL INSURANCE COST USING LINEAR REGRESSION" is a record of original work carried out by me under the guidance of S.YUVAKARTHIKA, COMPUTER SCIENCE, NPR ARTS AND SCIENCE COLLEGE, and that this project work has not formed the basis for the award of any degree, diploma, associateship, fellowship or similar title to any candidate in any university.

Place:

V.MADHU

Date: (Reg. No: C4S22373)

ACKNOWLEDGEMENT

I express my sincere gratitude to the Almighty for His blessings throughout the course of this project work.

I am deeply grateful to P.SARAVANAN, Principal, NPR ARTS AND SCIENCE COLLEGE, for providing the necessary facilities and encouragement to carry out this project successfully.

I extend my heartfelt thanks to G.VENKATESWARAN, Head of the Department, COMPUTER SCIENCE, for the constant support and valuable suggestions extended throughout the project period.

I would like to express my profound gratitude to my guide, S.YUVAKARTHIKA, COMPUTER SCIENCE, for the guidance, encouragement, and constructive feedback provided at every stage of this project, which contributed greatly to its successful completion.

I also thank all the faculty members and non-teaching staff of COMPUTER SCIENCE for their co-operation and support.

Finally, I thank my parents, family members, and friends for their constant encouragement, patience, and moral support throughout the successful completion of this project work.

V.MADHU

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION 13

1.1 About the Project 13
1.2 Project Description 13
1.3 Background of Medical Insurance Cost Prediction 13
1.4 Problem Statement 14
1.5 Need for the Project 14
1.6 Objectives of the Project 14
1.7 Scope of the Project 14
1.8 Importance of Medical Insurance Cost Estimation 15
1.9 Role of Machine Learning 15
1.10 Overview of Linear Regression 15
1.11 Advantages of the Proposed Application 15

CHAPTER 2 SYSTEM ANALYSIS 17

2.1 Feasibility Study 17
2.2 Existing System 17
2.3 Limitations of Existing System 18
2.4 Proposed System 18
2.5 Advantages of Proposed System 18
2.6 Problem Definition 18
2.7 Functional Requirements 18
2.8 Non-Functional Requirements 19

CHAPTER 3 SYSTEM ENVIRONMENT 20

3.1 Hardware Requirements 20
3.2 Software Requirements 20
3.3 Operating System 20
3.4 Programming Language 20
3.5 Development Environment 20
3.6–3.11 Python, NumPy, Pandas, Matplotlib, Scikit-learn, Tooling 21
3.12 Dataset 21
3.13 Dataset Attributes 22
3.14 Data Types 22
3.15 Software Specification 22

CHAPTER 4 SYSTEM DESIGN 23

4.1 Input Design 23
4.2 Output Design 23
4.3 System Architecture 23
4.4 System Workflow 24

4.5 Data Flow Diagram – Level 0 24
4.6 Data Flow Diagram – Level 1 24
4.7 ER Diagram 25
4.8 Use Case Diagram 26
4.9 Activity Diagram 26
4.10 Flowchart 27
4.11 Data Processing Flow 28
4.12 Machine Learning Workflow 29
CHAPTER 5 DATASET DESIGN 30
5.1 Dataset Description 30
5.2 Dataset Structure 30
5.3 Attribute Description 30
5.4 Data Dictionary 30
5.5 Input Features 30
5.6 Target Variable 31
5.7 Data Types 31
5.8 Missing Values 31
5.9 Data Cleaning 31
5.10 Data Preprocessing 31
5.11 Categorical Data Encoding 31
5.12 Feature Selection 31
5.13 Training Dataset 32
5.14 Testing Dataset 32
5.15 Dataset Relationship / Representation 32
CHAPTER 6 SYSTEM SPECIFICATION 33
6.1 Module Description 33
6.2 Dataset Input Module 33
6.3 Data Preprocessing Module 33
6.4 Feature Encoding Module 33
6.5 Feature Selection Module 34
6.6 Model Training Module 34
6.7 Linear Regression Prediction Module 34
6.8 User Input Module 35
6.9 Prediction Output Module 35
6.10 Result Visualization Module 35
6.11 Model Evaluation Module 35
CHAPTER 7 SYSTEM IMPLEMENTATION 37
7.1 Implementation Overview 37
7.2 Dataset Loading 37

7.3 Data Exploration 37
7.4 Data Cleaning 37
7.5 Data Preprocessing 37
7.6 Encoding Categorical Variables 38
7.7 Feature Selection 38
7.8 Train-Test Split 38
7.9 Linear Regression Model Creation 38
7.10 Model Training 38
7.11 Prediction 39
7.12 Model Evaluation 39
7.13 Error Calculation 39
7.14 Visualization 39
7.15 User Interface Implementation 39
7.16 Output Generation 40
7.17 Complete System Workflow 40
CHAPTER 8 SYSTEM TESTING 41
8.1 Testing Introduction 41
8.2 Testing Objectives 41
8.3 Unit Testing 41
8.4 Integration Testing 41
8.5 Functional Testing 41
8.6 Performance Testing 42
8.7 Validation Testing 42
8.8 User Acceptance Testing 42
8.9 Model Testing 42
8.10–8.12 Input Validation, Prediction & Error-Handling Test Table 42
CHAPTER 9 RESULTS AND DISCUSSION 44
9.1 Results Overview 44
9.2 Dataset Analysis 44
9.3 Feature Analysis 44
9.4 Training Results 46
9.5 Testing Results 46
9.6 Prediction Results 46
9.7 Model Performance 47
9.8 Regression Visualization 47
9.9 Actual vs Predicted Values 48
9.10 Error Analysis 48
9.11 Discussion of Results 48
9.12 Interpretation of Prediction 49

CHAPTER 10 CONCLUSION 50
CHAPTER 11 FUTURE ENHANCEMENT 51
CHAPTER 12 BIBLIOGRAPHY 52
12.1 Reference Books 52
12.2 Web References 52
APPENDIX 53
A. Screenshots 53
B. Dataset Sample 53
C. Sample Input 54
D. Sample Output 54
E. Model Output 54
F. Graphs / Charts 54
G. Important Source Code 54
H. Installation / Execution Steps 58
I. Sample Test Cases 59

LIST OF FIGURES

Fig. 4.1 System Architecture of the Medical Insurance Cost Estimation System 24
Fig. 4.2 Data Flow Diagram – Level 0 (Context Diagram) 24
Fig. 4.3 Data Flow Diagram – Level 1 25
Fig. 4.4 Entity-Attribute Representation of the Insurance Dataset 25
Fig. 4.5 Use Case Diagram 26
Fig. 4.6 Activity Diagram of the Prediction Process 27
Fig. 4.7 Flowchart of the Model Training Process 28
Fig. 4.8 Machine Learning Workflow 29
Fig. 9.1 Distribution of Estimated Insurance Charges in the Dataset 44
Fig. 9.2 Charges by Smoking Status 45
Fig. 9.3 Regression Coefficients by Feature 45
Fig. 9.3a Feature Correlation Matrix 46
Fig. 9.4 Actual vs. Predicted Insurance Charges (Test Set) 47
Fig. 9.5 Residual Plot for the Test Set 48
Fig. A.1 Medical Insurance Estimator — Web Interface (Default Values) 53

LIST OF TABLES

Table 2.1 Functional Requirements of the System 18
Table 2.2 Non-Functional Requirements of the System 19
Table 3.1 Minimum Hardware Requirements 20
Table 3.2 Software Requirements 20
Table 3.3 Dataset Attributes Used in the Project 22
Table 5.1 Data Dictionary for the Insurance Dataset 30
Table 7.1 Learned Model Parameters 38
Table 8.1 System Test Cases 42
Table 9.1 Model Evaluation Results on the Held-Out Test Set 46
Table 9.2 Sample Predictions on Illustrative Records 48
Table B.1 Sample Records from the Generated Dataset 53

ABSTRACT

Medical insurance charges vary widely between individuals based on personal and lifestyle factors such as age, body mass index (BMI), smoking status, and the number of dependents. Insurance providers must estimate these charges accurately in order to set premiums that are fair to policyholders while remaining financially sustainable for the insurer. Traditional approaches to this estimation are often based on fixed actuarial tables and manual underwriting rules, which can be slow to update and may not fully capture the relationships between multiple contributing factors. This project, titled "Estimate Medical Insurance Cost Using Linear Regression", presents a data-driven alternative that uses a supervised machine learning technique — multiple linear regression — to model and predict the estimated medical insurance charge for an individual from a small set of relevant input features.

The system consists of two co-operating components. The first is a Python-based model-training pipeline, built using the pandas, NumPy, and scikit-learn libraries, which prepares a dataset of individual records (age, BMI, smoker status, and number of children, together with the corresponding charge), splits the data into training and testing subsets, fits a Linear Regression model to the training subset, and evaluates the model on unseen test data using the Mean Squared Error (MSE) and the coefficient of determination (R^2). The second component is a browser-based interactive estimator, implemented using HTML, CSS, and JavaScript, which allows a user to adjust age, BMI, number of children, and smoking status using on-screen controls and instantly view an estimated insurance charge, together with a breakdown of how each factor contributes to the total.

The trained model achieved a coefficient of determination (R^2) of approximately 0.98 on the held-out test data, indicating that the linear model explains a large proportion of the variance in the simulated charge values, with smoking status identified as the single most influential factor. The project demonstrates the complete workflow of a regression-based estimation system — from data preparation and model training to evaluation and presentation of results through a user-facing interface — and provides a foundation that can be extended with real-world datasets, additional features, and more advanced algorithms in future work.

CHAPTER 1
INTRODUCTION

1.1 About the Project

"Estimate Medical Insurance Cost Using Linear Regression" is a machine-learning-based application that estimates the medical insurance charge likely to be incurred by an individual, based on a small set of personal and lifestyle attributes. The project combines a Python-based model-training pipeline with an interactive web-based front end so that a user can supply basic details — age, body mass index (BMI), number of children, and smoking status — and immediately view an estimated insurance cost along with a breakdown of the contribution made by each factor.

The core prediction technique used in the project is multiple linear regression, a well-established supervised learning algorithm that models the relationship between a set of independent (input) variables and a dependent (output) variable as a weighted linear combination. The project was implemented using the Python programming language together with the pandas, NumPy, and scikit-learn libraries for data handling, model training, and evaluation, and using HTML, CSS, and JavaScript for the interactive estimation interface.

1.2 Project Description

The project is organised around two co-operating parts. The first part is a data preparation and model-training script (insurance_model.py) that constructs a dataset of individual records, each containing age, BMI, smoker status, and number of children as input features and the corresponding insurance charge as the target value; splits the dataset into training and testing subsets; fits a Linear Regression model using scikit-learn; evaluates the trained model on the held-out test subset using standard regression metrics; and persists the trained model to disk using joblib so that it can be reloaded without retraining.

The second part is a client-side web application (index.html, style.css, and script.js) that presents an interactive form in which the user can adjust age, BMI, and number of children using range sliders and toggle smoking status using a switch control. As the user changes any input, the application recalculates the estimated charge in real time using a linear formula whose coefficients mirror the relationship learned during model development, and displays the resulting estimate together with a line-by-line breakdown of the amount contributed by each factor.

1.3 Background of Medical Insurance Cost Prediction

Health insurance premiums are conventionally determined through actuarial analysis, in which historical claims data is used to build risk tables segmented by broad categories such as age band, sex, and known health conditions. While this approach has been used successfully by insurers for decades, it relies on manually maintained rules and pre-defined categories that can be

slow to revise and that may not capture more subtle, continuous relationships between contributing factors — for example, the way in which the effect of BMI on cost may itself depend on whether an individual smokes.

The growth of affordable computing power and freely available machine-learning libraries has made it practical to apply statistical learning techniques directly to historical or simulated policyholder data in order to estimate expected charges. Regression-based techniques, in particular, are attractive for this purpose because they produce a continuous numeric estimate, are computationally inexpensive to train and evaluate, and — in the case of linear regression — yield coefficients that are directly interpretable as the marginal effect of each input feature on the predicted charge.

1.4 Problem Statement

Given a set of personal and lifestyle attributes for an individual — specifically age, BMI, number of children, and smoking status — the problem addressed by this project is to estimate the medical insurance charge that individual is likely to incur, using a model trained on historical (in this implementation, simulated) records that relate these attributes to an observed charge. The estimation must be fast enough to compute interactively as a user adjusts the input values, and the underlying model must be evaluable so that its accuracy can be quantified and reported.

1.5 Need for the Project

A tool of this kind is useful for several reasons. It allows an individual to obtain an immediate, approximate estimate of their expected insurance cost without needing to consult an agent or complete a formal application. It allows the relative influence of different lifestyle factors — most notably smoking — to be demonstrated clearly, which has value both as a decision-support tool and as an educational illustration of how regression models work. It also serves as a compact, complete example of an end-to-end machine-learning workflow: data preparation, model training, evaluation, and presentation of results through a user interface, making it a suitable academic project for demonstrating applied machine-learning and web-development skills together.

1.6 Objectives of the Project

- To construct a labelled dataset relating age, BMI, smoker status, and number of children to a medical insurance charge.
- To train a multiple linear regression model on this dataset using scikit-learn.
- To evaluate the trained model using appropriate regression metrics (Mean Squared Error and R-squared).
- To determine the relative contribution of each input feature to the predicted charge.
- To provide an interactive, browser-based interface through which a user can supply input values and view an estimated charge along with a per-factor breakdown.
- To document the complete workflow so that the project can be extended with real-world data or additional features in the future.

1.7 Scope of the Project

The scope of the present implementation covers four input features — age, BMI, smoker status, and number of children — and a single continuous target, the estimated insurance charge. The model-training component operates on a programmatically generated dataset of 1,000 records with a fixed random seed, which allows the results reported in this document to be reproduced exactly. The web-based interface reproduces the linear relationship interactively without requiring a live connection to the Python model at run time. The scope does not, in the present implementation, include attributes such as sex, geographic region, or pre-existing medical conditions, nor does it include server-side deployment of the trained model; these are identified as directions for future enhancement in Chapter 11.

1.8 Importance of Medical Insurance Cost Estimation

Accurate cost estimation benefits both the insured individual and the insurer. For the individual, an early estimate assists with financial planning and helps set realistic expectations before a formal quotation is obtained. For the insurer, a systematic, data-driven estimation approach supports consistent and defensible pricing, helps identify the factors that most strongly influence cost, and can be updated as new data becomes available, in a way that is more transparent than opaque manual rules.

1.9 Role of Machine Learning

Machine learning provides a means of learning the relationship between input attributes and an outcome directly from data, rather than relying on manually specified rules. In this project, the learning task is a supervised regression problem: the model is given a set of labelled training examples (input feature vectors paired with an observed charge) and learns a function that maps new, previously unseen feature vectors to a predicted charge. Once trained, the model can generate predictions for new individuals in constant time, which makes it well suited to interactive applications.

1.10 Overview of Linear Regression

Linear regression is one of the most widely used supervised learning algorithms for problems in which the target variable is continuous. In its multiple-variable form, the model assumes that the target value y can be approximated as a weighted sum of the input features x1, x2, …, xn, plus a constant offset:

$$y = b0 + b1x1 + b2x2 + \ldots + bnxn$$

where b0 is the intercept, and b1 through bn are the coefficients associated with each input feature. During training, the algorithm chooses the coefficients that minimise the sum of squared differences between the predicted values and the actual observed values across the training set (the ordinary least squares criterion). Because the resulting model is a simple linear equation, each coefficient can be interpreted directly as the expected change in the target value for a one-unit

change in the corresponding feature, holding all other features constant — a property that this project uses directly when explaining, to the end user, how much each factor contributes to the estimated charge.

1.11 Advantages of the Proposed Application

- Provides an instant, interactive estimate as the user changes input values, with no page reloads or server round-trips required.
- Uses an interpretable model whose coefficients directly explain the contribution of each input feature.
- Is built entirely from freely available, open-source technologies (Python, scikit-learn, HTML, CSS, and JavaScript).
- Separates model development from presentation, allowing the trained model to be retrained or improved independently of the interface.
- Provides a reproducible, documented baseline that can be extended with real datasets and additional features.

CHAPTER 2
SYSTEM ANALYSIS

2.1 Feasibility Study

Before development began, a feasibility study was carried out to confirm that the proposed system could be built with the tools, time, and skills available, and that the resulting application would be genuinely useful. The study examined technical, economic, operational, and schedule feasibility, as summarised in the following subsections.

2.1.1 Technical Feasibility

The project relies entirely on mature, freely available open-source technologies. Python, together with pandas, NumPy, and scikit-learn, provides all of the functionality required for data preparation, model training, and evaluation. HTML, CSS, and JavaScript are supported natively by every modern web browser and require no additional runtime or plugin to execute the interactive estimator. No specialised or proprietary hardware is required; the project can be developed and run on any standard desktop or laptop computer. The technical feasibility of the project is therefore considered high.

2.1.2 Economic Feasibility

All of the software used in the project — the Python interpreter, the pandas, NumPy, scikit-learn, and joblib libraries, and standard web browsers — is available free of charge under open-source or freely redistributable licences. No paid cloud services, proprietary datasets, or licensed software components are required for the current scope of the project. The only cost associated with the project is the developer's time, and consequently the project is considered economically feasible.

2.1.3 Operational/Behavioral Feasibility

The application requires no specialised training to operate. The web-based estimator uses familiar interface elements — sliders and a toggle switch — that most users are already comfortable with from other websites and applications, and the result is displayed immediately without any additional steps. The model-training script is intended for a developer or student audience and follows a conventional, well-documented scikit-learn workflow. The system is therefore considered operationally feasible for its intended audience.

2.1.4 Schedule Feasibility

The scope of the project — four input features, a single regression model, and a single-page interactive interface — is deliberately kept compact so that it can be completed, tested, and documented within a typical academic project timeframe. The modular separation between the model-training script and the web interface also allowed each part to be developed and verified independently, reducing the overall development schedule.

2.2 Existing System

In the absence of a tool such as the one proposed here, an individual who wishes to estimate their medical insurance cost typically has two options. The first is to consult publicly published, broad-category rate tables provided by an insurer, which group individuals into coarse bands (for example, by age range and smoker/non-smoker status) and provide a single indicative premium for each band. The second is to complete a full application or request a formal quotation, which requires submitting detailed personal information and waiting for a manual or semi-automated underwriting process to return a result.

2.3 Limitations of Existing System

- Rate tables based on broad categories do not reflect the continuous nature of factors such as age and BMI, and can under- or over-estimate cost for individuals near a category boundary.
- Manual or table-based estimation does not make clear how much each individual factor (age, BMI, smoking, number of children) contributes to the final figure.
- Obtaining a formal quotation is comparatively slow, and is not suited to quickly comparing the effect of changing a single lifestyle factor, such as quitting smoking.
- Existing published tables are typically static and are not easily re-derived or updated as new data becomes available.

2.4 Proposed System

The proposed system addresses these limitations by training a multiple linear regression model directly on a labelled dataset of individual records, so that the relationship between each continuous input feature and the resulting charge is learned rather than manually categorised. The trained model's coefficients are then used to power an interactive web interface in which a user can adjust age, BMI, number of children, and smoker status using simple controls and receive an estimated charge instantly, together with a breakdown showing exactly how much each factor contributes to the total.

2.5 Advantages of Proposed System

- Produces a continuous, individualised estimate rather than a coarse category-based figure.
- Clearly itemises the contribution of each input factor to the final estimated charge.
- Updates the estimate instantly as the user changes any input, without a server round-trip.
- Is based on a model whose accuracy can be objectively measured and reported (Mean Squared Error and R-squared).
- Can be retrained on new or real-world data without changing the structure of the application.

2.6 Problem Definition

The problem to be solved is formally a supervised regression problem: given a training set of records $\{(x(i), y(i))\}$ for $i = 1 \ldots n$, where each $x(i)$ is a feature vector containing age, BMI, smoker

status, and number of children, and y(i) is the corresponding observed insurance charge, learn a function f(x) = b0 + b1x1 + b2x2 + b3x3 + b4x4 that minimises the error between f(x(i)) and y(i) across the training set, and that generalises well to previously unseen feature vectors.

2.7 Functional Requirements

Table 2.1: Functional Requirements of the System

ID	Requirement	Description
FR-1	Dataset Preparation	The system shall construct a dataset containing age, BMI, smoker status, children, and charge for a set of individual records.
FR-2	Model Training	The system shall train a Linear Regression model using the prepared dataset.
FR-3	Model Evaluation	The system shall compute and report the Mean Squared Error and R-squared of the trained model on a held-out test set.
FR-4	Model Persistence	The system shall save the trained model to disk so that it can be reloaded without retraining.
FR-5	Interactive Input	The system shall allow a user to specify age, BMI, number of children, and smoker status through on-screen controls.
FR-6	Real-Time Prediction	The system shall compute and display an estimated insurance charge immediately whenever an input value changes.
FR-7	Result Breakdown	The system shall display the contribution of each input factor to the total estimated charge.

2.8 Non-Functional Requirements

Table 2.2: Non-Functional Requirements of the System

ID	Requirement	Description
NFR-1	Usability	The interface shall be simple enough to operate without instructions, using familiar slider and toggle controls.
NFR-2	Performance	The estimated charge shall be recalculated and displayed within a fraction of a second of any input change.
NFR-3	Portability	The web interface shall run in any modern browser without additional installation.
NFR-4	Maintainability	The model-training script and the web interface shall be kept as separate, independently modifiable components.
NFR-5	Reproducibility	The dataset generation and model training shall use a fixed random seed so that results can be reproduced exactly.

CHAPTER 3
SYSTEM ENVIRONMENT

3.1 Hardware Requirements

Table 3.1: Minimum Hardware Requirements

Component	Specification
Processor	Intel Core i3 (or equivalent) and above
RAM	4 GB and above
Hard Disk	20 GB free space and above
Display	Standard colour monitor (1366 x 768 or higher)
Input Devices	Keyboard and mouse

3.2 Software Requirements

Table 3.2: Software Requirements

Component	Specification
Operating System	Windows 10 / 11, Linux, or macOS
Programming Language	Python 3.x
Front-End Technologies	HTML5, CSS3, JavaScript (ES6)
Python Libraries	pandas, NumPy, scikit-learn, joblib
Development Environment	Any Python-capable code editor / IDE and a modern web browser
Model Persistence Format	joblib (.joblib)

3.3 Operating System

The project was developed and tested in an environment supporting Python 3 and a modern web browser; it does not depend on any operating-system-specific feature and therefore runs equally on Windows, Linux, or macOS.

3.4 Programming Language

The model-training component of the project is implemented in Python, a high-level, interpreted programming language widely used for data analysis and machine learning because of its concise syntax and extensive ecosystem of scientific libraries. The interactive front end is implemented in standard JavaScript (ES6), which runs natively inside the browser without requiring any build step or external runtime.

3.5 Development Environment

Development was carried out using a standard code editor together with a Python interpreter for the model-training script, and a web browser for viewing and testing the

HTML/CSS/JavaScript front end. No external database server, web server, or cloud service is required to run either component; the front end can be opened directly as a static HTML file, and the Python script can be executed directly from the command line.

3.6 Python

Python was selected for the data-preparation and model-training component because of its readability and because it provides direct access, through well-maintained libraries, to every stage of the machine-learning workflow used in this project: data generation and manipulation (pandas, NumPy), model training and evaluation (scikit-learn), and model persistence (joblib).

3.7 NumPy

NumPy provides efficient array-based numerical computation and is used in this project to generate the synthetic input features (age, BMI, smoker status, and number of children) using its random-number generation functions, and to add controlled random noise to the simulated charge values so that they resemble real-world variability rather than an exact deterministic formula.

3.8 Pandas

Pandas provides the DataFrame data structure used throughout the project to hold the tabular dataset of individual records. It is used to assemble the generated feature and target arrays into a single structured dataset, to select the input feature columns and the target column prior to training, and to construct single-row DataFrames representing a new individual when generating an example prediction.

3.9 Matplotlib

Matplotlib was used during the analysis and documentation stage of the project to visualise the trained model's behaviour and performance — including the relationship between actual and predicted charges, the distribution of charges in the dataset, and the magnitude of each feature's regression coefficient — in support of the results discussed in Chapter 9 of this report.

3.10 Scikit-learn

Scikit-learn is the primary machine-learning library used in this project. It supplies the train_test_split function used to divide the dataset into training and testing subsets, the LinearRegression estimator used to fit the regression model, and the mean_squared_error and r2_score functions used to evaluate the trained model's performance on the test subset.

3.11 Development / Execution Tooling

The model-training script (insurance_model.py) is executed directly using the Python interpreter from the command line, without requiring a notebook environment. The web-based estimator consists of static HTML, CSS, and JavaScript files that can be opened directly in any

modern browser, or served through any basic static file server, without requiring a dedicated application server or database.

3.12 Dataset

The dataset used to train and evaluate the model in the current implementation is generated programmatically within insurance_model.py using NumPy's random-number generator with a fixed seed (numpy.random.seed(42)), which ensures that the same 1,000-record dataset — and consequently the same trained model and evaluation results — is produced every time the script is executed. Age is drawn as an integer between 18 and 65, BMI as a continuous value between 15 and 40, smoker status as a binary indicator (0 or 1), and number of children as an integer between 0 and 3. The corresponding charge for each record is computed as a base amount plus weighted contributions from each feature, with Gaussian random noise added to simulate natural variability. This approach was adopted because it produces a fully reproducible dataset whose true underlying relationship is known, which is useful for demonstrating and verifying the regression workflow; Chapter 11 identifies replacing this synthetic dataset with a real-world insurance dataset as a direction for future enhancement.

3.13 Dataset Attributes

Table 3.3: Dataset Attributes Used in the Project

Attribute	Type	Description
age	Integer	Age of the individual in years (18–65)
bmi	Float	Body Mass Index of the individual (15–40)
smoker	Integer (0/1)	Smoking status: 0 = non-smoker, 1 = smoker
children	Integer	Number of children/dependents (0–3)
charges	Float	Target variable: simulated medical insurance charge

3.14 Data Types

All input features used by the model are numeric: age and number of children are represented as integers, BMI as a floating-point value, and smoker status as a binary integer (0 or 1), which allows it to be used directly in the linear regression equation without additional one-hot encoding. The target variable, charges, is a continuous floating-point value.

3.15 Software Specification

- Python 3.x interpreter with pip package manager
- pandas — DataFrame-based data handling
- NumPy — numerical array operations and random data generation
- scikit-learn — train/test splitting, Linear Regression model, evaluation metrics
- joblib — saving and loading the trained model
- HTML5 / CSS3 — structure and styling of the estimator interface

- JavaScript (ES6) — client-side calculation and dynamic updates

CHAPTER 4
SYSTEM DESIGN

4.1 Input Design

The input design of the system is centred on four controls presented in the web interface: a range slider for age (18–100), a range slider for BMI (15–50, in steps of 0.1), a range slider for number of children (0–10), and a toggle switch for smoker status. Each slider is bound to a numeric label that updates live as the control is moved, so that the user can see the exact value being submitted before and while the estimate is recalculated. This design was chosen because it removes the possibility of invalid free-text input (for example, non-numeric age values) by constraining every input to a valid range through the control itself.

4.2 Output Design

The output is presented in a dedicated results panel showing the estimated charge as a large, prominently styled monetary figure, together with a smaller breakdown section listing the base charge and the dollar contribution of each of the four input factors (age, BMI, smoker status, and children). The estimated amount is animated smoothly from its previous value to the new value whenever an input changes, which gives the user clear visual feedback that a recalculation has taken place.

4.3 System Architecture

The system is composed of two loosely coupled components that share the same underlying linear relationship: a Python-based model-training pipeline that produces a trained Linear Regression model together with its learned coefficients, and a browser-based presentation layer that reproduces the estimation formula in JavaScript for instant, client-side calculation. The two components are not connected by a live network call; the presentation layer's coefficients were set to correspond to the relationship used to generate the training data, so that the interactive front end mirrors the behaviour of the trained model without requiring a backend server. This architecture is illustrated in Figure 4.1.

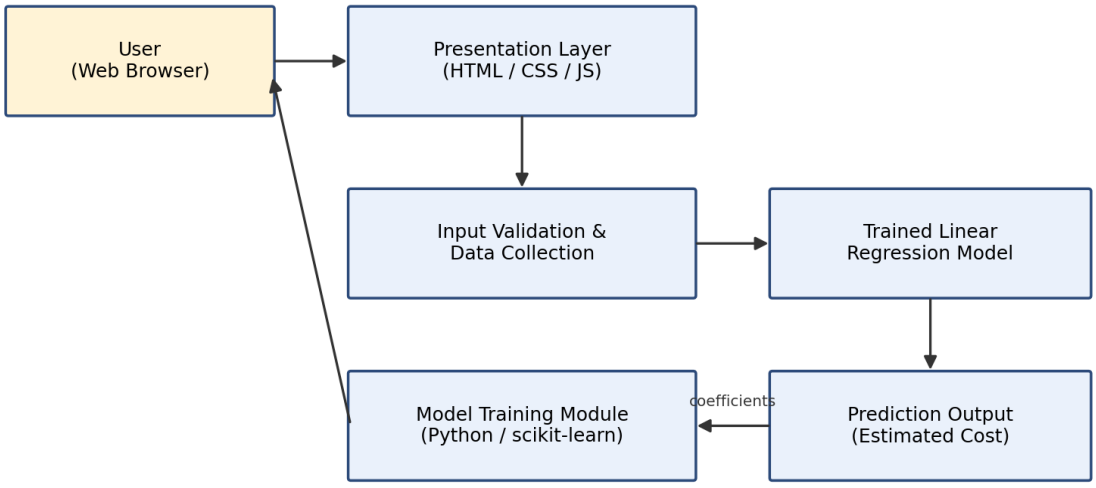


Fig. 4.1: System Architecture of the Medical Insurance Cost Estimation System

4.4 System Workflow

At a high level, the system workflow proceeds as follows: the model-training script generates or loads the dataset, cleans and prepares it, splits it into training and testing subsets, fits a Linear Regression model, evaluates its accuracy, and saves the trained model. Independently, the web interface collects user input through its on-screen controls, applies the same linear formula, and displays the resulting estimate together with a breakdown. This overall data flow is represented as a Data Flow Diagram (DFD) in Sections 4.5 and 4.6.

4.5 Data Flow Diagram – Level 0

The Level 0 (context) DFD treats the entire system as a single process that receives personal and lifestyle attributes from the user and returns an estimated insurance charge, as shown in Figure 4.2.

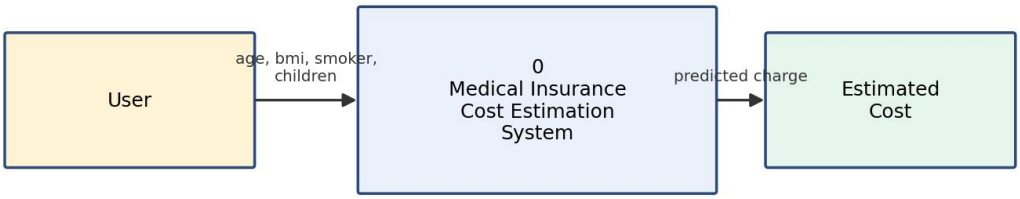


Fig. 4.2: Data Flow Diagram – Level 0 (Context Diagram)

4.6 Data Flow Diagram – Level 1

The Level 1 DFD decomposes the system into its constituent processes: capturing user input, validating and preprocessing that input, applying the trained Linear Regression model (using the coefficients derived from the training dataset), and generating and displaying the final prediction, as shown in Figure 4.3.

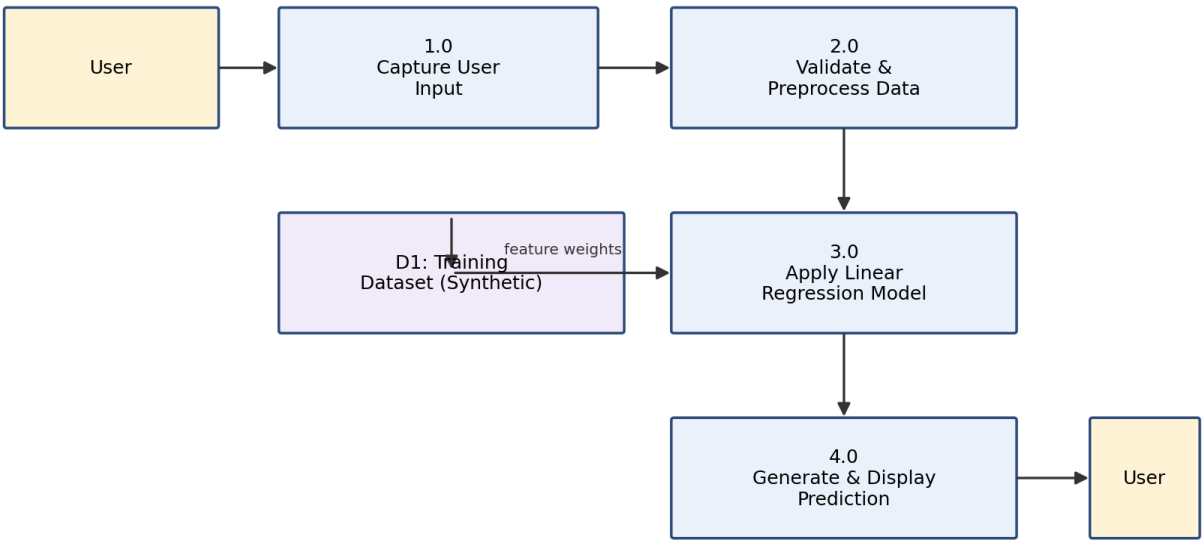


Fig. 4.3: Data Flow Diagram – Level 1

4.7 ER Diagram

Because the project does not use a relational database, Section 4.7 instead presents an entity-attribute representation of the single dataset entity used throughout the system, INSURANCE_RECORD, which groups the four input attributes and the target attribute that together define one training example, as shown in Figure 4.4.

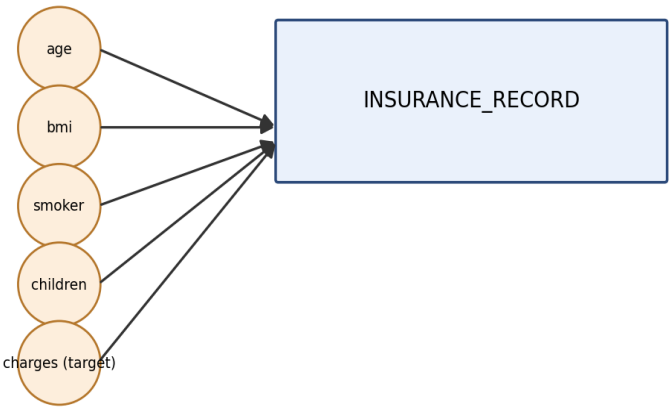


Fig. 4.4: Entity-Attribute Representation of the Insurance Dataset

4.8 Use Case Diagram

The use case diagram in Figure 4.5 shows the single actor of the system — the User — and the three principal use cases available to that actor: entering personal and lifestyle details, viewing the estimated insurance cost, and viewing the breakdown of how that cost was calculated.

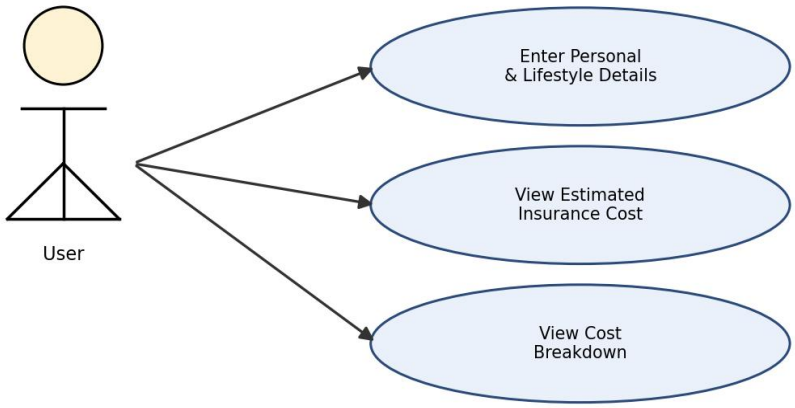


Fig. 4.5: Use Case Diagram

4.9 Activity Diagram

The activity diagram in Figure 4.6 traces the sequence of activities that occur when a user interacts with the estimator: opening the web interface, adjusting one or more input controls, the system reading the updated values, computing the estimated charge using the regression formula, and displaying the updated result and breakdown.

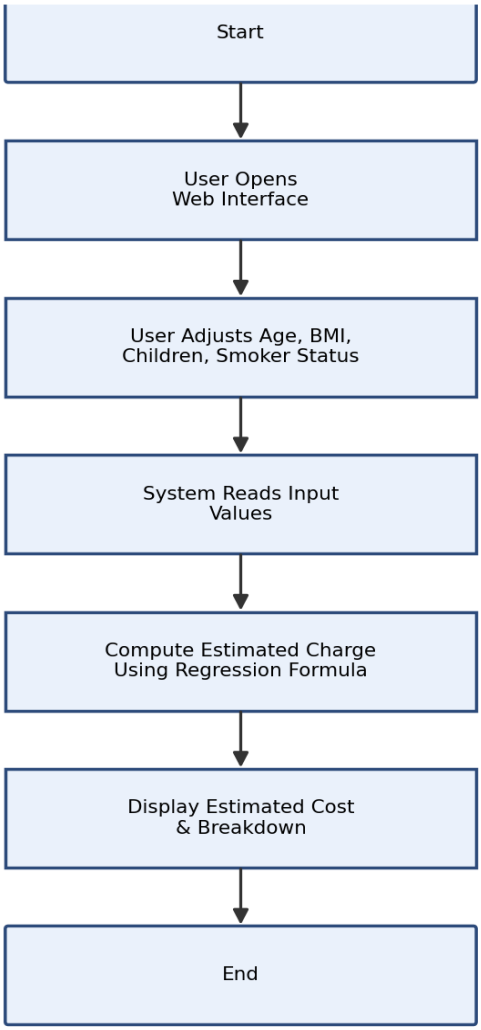


Fig. 4.6: Activity Diagram of the Prediction Process

4.10 Flowchart

The flowchart in Figure 4.7 describes the offline process followed by the model-training script, from data generation through to saving the trained model.

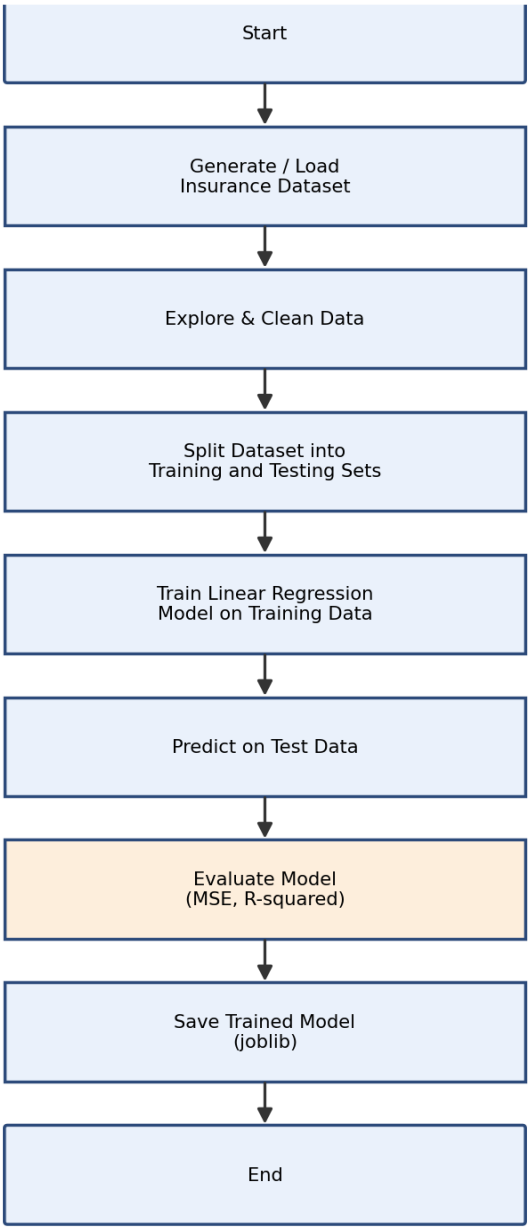


Fig. 4.7: Flowchart of the Model Training Process

4.11 Data Processing Flow

Data flows through the model-training pipeline in a fixed sequence: raw feature arrays are generated using NumPy, assembled into a single pandas DataFrame, separated into an input feature matrix X and a target vector y, and split into training and testing subsets using an 80:20 ratio with a fixed random state so that the split is reproducible. The training subset is used exclusively to fit the model, and the testing subset is reserved exclusively for evaluating its accuracy on unseen data, which avoids optimistic bias in the reported metrics.

4.12 Machine Learning Workflow

Figure 4.8 summarises the end-to-end machine learning workflow followed in this project, from the initial synthetic dataset through preprocessing, splitting, training, evaluation, and finally deployment of the learned relationship in the web-based presentation layer.

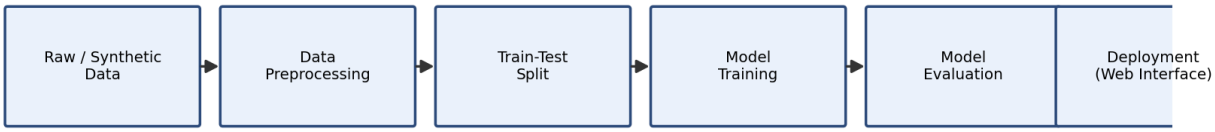


Fig. 4.8: Machine Learning Workflow

CHAPTER 5
DATASET DESIGN

5.1 Dataset Description

Since the project does not use a relational database, this chapter documents the design of the dataset that plays the equivalent role: the structured, tabular collection of records used to train and evaluate the Linear Regression model. The dataset is generated programmatically inside insurance_model.py and consists of 1,000 individual records, each describing one simulated policyholder in terms of four input features and one target value.

5.2 Dataset Structure

The dataset is held in memory as a pandas DataFrame with five columns — age, bmi, smoker, children, and charges — and 1,000 rows, one per simulated individual. Each row is independent of every other row; there are no relationships between records, and consequently no need for a multi-table or relational structure.

5.3 Attribute Description

Age is generated as a uniformly distributed random integer between 18 and 65. BMI is generated as a uniformly distributed random floating-point value between 15 and 40. Smoker status is generated as a random binary value (0 or 1) with equal probability. Number of children is generated as a random integer between 0 and 3. The target attribute, charges, is computed as the sum of a fixed base amount (2,000), a term proportional to age (weight 250), a term proportional to BMI (weight 150), a term of 20,000 applied only when the individual is a smoker, a term proportional to the number of children (weight 500), and Gaussian random noise with a mean of 0 and a standard deviation of 1,500, which introduces natural-looking variability around the underlying linear relationship.

5.4 Data Dictionary

Table 5.1: Data Dictionary for the Insurance Dataset

Attribute Name	Data Type	Description	Role
age	Integer	Age of the individual in years	Input Feature
bmi	Float	Body Mass Index of the individual	Input Feature
smoker	Integer (0/1)	Smoking status of the individual	Input Feature
children	Integer	Number of children/dependents	Input Feature
charges	Float	Simulated medical insurance charge	Target Variable

5.5 Input Features

The four input features — age, bmi, smoker, and children — form the feature matrix X supplied to the model. All four are purely numeric, which means that, unlike categorical attributes such as sex or region, none of them requires additional preprocessing (such as one-hot encoding) before being used directly in the linear regression equation.

5.6 Target Variable

The target variable, charges, is a continuous, strictly positive floating-point value representing the simulated total medical insurance cost associated with an individual record. It is the value the Linear Regression model is trained to predict.

5.7 Data Types

Table 5.1 above lists the data type of every attribute in the dataset. All attributes are numeric (integer or floating-point), which keeps the preprocessing pipeline simple and avoids the need for categorical encoding in the current implementation.

5.8 Missing Values

Because the dataset is generated programmatically using NumPy's random-number functions rather than collected from an external source, it contains no missing, null, or malformed values by construction. Every one of the 1,000 records has a fully populated value for each of the five attributes. This is noted here because it differs from what would typically be encountered with a real-world dataset, where missing-value handling (imputation or row removal) would ordinarily be a necessary preprocessing step; this is discussed further as a future enhancement in Chapter 11.

5.9 Data Cleaning

As a consequence of the synthetic generation process described above, no data-cleaning steps (such as removing duplicate records, correcting invalid values, or handling outliers) were required for this dataset. The generation ranges were chosen deliberately (for example, age between 18 and 65, BMI between 15 and 40) to remain within realistic bounds, so no out-of-range values are present.

5.10 Data Preprocessing

The only preprocessing step applied prior to training is the separation of the DataFrame into an input feature matrix X (containing the age, bmi, smoker, and children columns) and a target vector y (containing the charges column), followed by an 80:20 train-test split performed using scikit-learn's train_test_split function with a fixed random_state of 42 to ensure reproducibility.

5.11 Categorical Data Encoding

The smoker attribute, although conceptually categorical (smoker / non-smoker), is generated directly as a numeric binary indicator (0 or 1), and therefore no additional encoding step such as

one-hot or label encoding is required in the current implementation. Should additional categorical attributes (for example, region or sex) be introduced in a future version of the project, one-hot encoding would be the natural extension, as discussed in Chapter 11.

5.12 Feature Selection

All four generated input features — age, bmi, smoker, and children — are used in the model without any feature-selection or dimensionality-reduction step, since the dataset was constructed specifically to include only attributes known to influence the simulated charge. The relative importance of each retained feature is instead assessed after training, by examining the magnitude of its learned regression coefficient, as discussed in Chapter 9.

5.13 Training Dataset

The training subset comprises 800 records (80% of the 1,000 generated records), selected automatically by scikit-learn's train_test_split function. This subset is used exclusively to fit the Linear Regression model's coefficients.

5.14 Testing Dataset

The testing subset comprises the remaining 200 records (20% of the 1,000 generated records). This subset is not used during training and is reserved exclusively for evaluating the trained model's predictive accuracy on data it has not previously seen, as reported in Chapter 9.

5.15 Dataset Relationship / Representation

As noted in Section 5.2, the dataset consists of a single flat table with no relationships to other tables, and therefore no entity-relationship diagram beyond the single-entity attribute representation already presented in Figure 4.4 (Chapter 4) is required for the current implementation.

CHAPTER 6
SYSTEM SPECIFICATION

6.1 Module Description

The system is organised into a small number of clearly separated modules, each responsible for one stage of the overall workflow described in Chapter 4. The Python-side modules (Sections 6.2–6.6 and 6.11) are implemented within insurance_model.py, and the client-side modules (Sections 6.7–6.10) are implemented within the index.html, style.css, and script.js files. Each module is described below in terms of its purpose, input, processing, output, and role in the system.

6.2 Dataset Input Module

Purpose: To construct the labelled dataset used to train and evaluate the model.

Input: A fixed random seed (42) and the number of records to generate (1,000).

Processing: Uses NumPy's random-number generation functions to produce arrays of age, BMI, smoker status, and number of children, and computes a corresponding charges value for each record using the base-plus-weighted-features formula described in Chapter 5.

Output: A pandas DataFrame containing 1,000 rows and the five columns: age, bmi, smoker, children, charges.

Role in the Overall System: Supplies the raw dataset consumed by every subsequent module in the training pipeline.

6.3 Data Preprocessing Module

Purpose: To prepare the generated dataset for use by the Linear Regression estimator.

Input: The full DataFrame produced by the Dataset Input Module.

Processing: Separates the DataFrame into an input feature matrix X (age, bmi, smoker, children) and a target vector y (charges).

Output: Feature matrix X and target vector y, ready for splitting.

Role in the Overall System: Provides the structured inputs required by scikit-learn's train_test_split and LinearRegression classes.

6.4 Feature Encoding Module

Purpose: To ensure that every input feature is in a numeric form suitable for direct use in the regression equation.

Input: The feature matrix X.

Processing: Confirms that all four features (age, bmi, smoker, children) are already numeric; the smoker attribute is generated directly as a 0/1 indicator, so no additional one-hot or label encoding is required in the current implementation.

Output: A fully numeric feature matrix.

Role in the Overall System: Removes the need for a separate categorical-encoding step in the present dataset, while leaving the pipeline structured so that an encoding step could be inserted here if categorical attributes were introduced in future.

6.5 Feature Selection Module

Purpose: To determine which attributes are used as inputs to the model.

Input: The four generated attributes: age, bmi, smoker, children.

Processing: All four attributes are retained, since each was included specifically because it contributes to the simulated target value; no statistical feature-elimination step is applied in the current implementation.

Output: The finalised list of input features used by the Model Training Module.

Role in the Overall System: Defines the dimensionality of the regression problem (four input features, one target).

6.6 Model Training Module

Purpose: To fit a Linear Regression model to the training data.

Input: The training subset of the feature matrix X_train and target vector y_train, produced by an 80:20 train_test_split with random_state=42.

Processing: Creates a scikit-learn LinearRegression estimator and calls its fit() method on X_train and y_train, which computes the intercept and per-feature coefficients that minimise the sum of squared residuals on the training data.

Output: A trained LinearRegression model object with learned coefficients and intercept.

Role in the Overall System: Produces the core predictive model used by the Prediction and Evaluation modules, and persisted by the Model Persistence step.

6.7 Linear Regression Prediction Module

Purpose: To produce a predicted insurance charge for a given set of input feature values.

Input: A feature vector (age, bmi, smoker, children), either from the held-out test set or from a newly constructed example record.

Processing: Applies the trained model's predict() method, which evaluates the linear equation $y = b0 + b1 \cdot age + b2 \cdot bmi + b3 \cdot smoker + b4 \cdot children$ using the learned coefficients.

Output: A single predicted charge value (in the Python pipeline) or an array of predicted values (when applied to the test set).

Role in the Overall System: Generates the predictions used both for model evaluation (Section 6.11) and for the illustrative example predictions printed by the script.

6.8 User Input Module

Purpose: To capture age, BMI, number of children, and smoker status from the user through the web interface.

Input: User interaction with the range sliders (age, bmi, children) and the toggle switch (smoker) defined in index.html.

Processing: JavaScript event listeners attached to each control (input and change events) read the current value of the control whenever it changes.

Output: The current numeric values of age, bmi, children, and the boolean smoker state.

Role in the Overall System: Provides the live input values consumed by the Prediction Output Module on the client side.

6.9 Prediction Output Module

Purpose: To compute and display the estimated insurance charge in the browser.

Input: The current age, bmi, children, and smoker values captured by the User Input Module.

Processing: Evaluates the same linear formula used to generate the training data (base charge plus weighted age, BMI, smoker, and children terms) directly in JavaScript, and animates the displayed amount smoothly from its previous value to the newly computed value.

Output: An updated estimated charge shown in the results panel of the web page.

Role in the Overall System: Delivers the primary output of the interactive estimator to the user in real time.

6.10 Result Visualization Module

Purpose: To help the user understand how the total estimated charge was derived.

Input: The current age, bmi, children, and smoker values, and the fixed per-feature weights used in the formula.

Processing: Computes the dollar contribution of the base amount and of each of the four factors individually, formats each as a currency string, and writes them into the calculation-details section of the results panel.

Output: A itemised, human-readable breakdown of the estimated charge, displayed beneath the total.

Role in the Overall System: Provides transparency and interpretability, allowing the user to see the effect of each individual input rather than only the final total.

6.11 Model Evaluation Module

Purpose: To quantify how accurately the trained model predicts charges on data it has not seen during training.

Input: The held-out test subset (X_test, y_test) and the predictions generated for it by the trained model.

Processing: Computes the Mean Squared Error (mean_squared_error) and the coefficient of determination (r2_score) by comparing the predicted values to the actual test-set charges.

Output: Two summary metrics — MSE and R-squared — printed to the console and reported in Chapter 9 of this document.

Role in the Overall System: Provides the objective measure of model quality referenced throughout the results and testing chapters of this report.

CHAPTER 7
SYSTEM IMPLEMENTATION

7.1 Implementation Overview

This chapter describes how each stage of the design presented in Chapters 4 through 6 was implemented in code. The implementation is divided between the model-training script, insurance_model.py, which performs data generation, training, and evaluation, and the client-side files (index.html, style.css, script.js), which implement the interactive estimator. The complete listings for both are provided in the Appendix; this chapter explains the key steps and the reasoning behind them.

7.2 Dataset Loading

The dataset used in this implementation is generated rather than loaded from an external file. numpy.random.seed(42) fixes the random seed so that every execution of the script produces an identical dataset, which is essential for the reproducibility of the results reported in this document. One thousand values are then generated for each of the four input features using NumPy's randint and uniform functions.

```
np.random.seed(42)
n_samples = 1000

age = np.random.randint(18, 65, n_samples)
bmi = np.random.uniform(15, 40, n_samples)
smoker = np.random.randint(0, 2, n_samples)
children = np.random.randint(0, 4, n_samples)
```

7.3 Data Exploration

Following generation, the script prints the first five rows of the resulting DataFrame using data.head(), which allows a quick visual check that the generated values fall within the expected ranges before proceeding to training. This step corresponds to the exploratory check typically performed on any dataset before it is used to train a model.

7.4 Data Cleaning

As discussed in Chapter 5, the synthetically generated dataset contains no missing values, duplicate records, or out-of-range values by construction, and therefore no explicit data-cleaning code is required in the current implementation.

7.5 Data Preprocessing

The target value, charges, is computed as a linear combination of the four features plus Gaussian noise, and the resulting values are assembled into a single pandas DataFrame:

```
charges = 2000 + (age * 250) + (bmi * 150) + (smoker * 20000) \
          + (children * 500) + np.random.normal(0, 1500, n_samples)
```

```
data = pd.DataFrame({
    'age': age, 'bmi': bmi, 'smoker': smoker,
    'children': children, 'charges': charges
})
```

7.6 Encoding Categorical Variables

As noted in Chapter 6, the smoker attribute is generated directly in numeric (0/1) form, so no additional categorical-encoding step is required before this attribute can be used in the regression equation.

7.7 Feature Selection

The feature matrix X and target vector y are extracted from the DataFrame as follows:

```
X = data[['age', 'bmi', 'smoker', 'children']]
y = data['charges']
```

7.8 Train-Test Split

The dataset is divided into training and testing subsets using an 80:20 split, with the random_state parameter fixed at 42 so that the same split is produced on every run:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

7.9 Linear Regression Model Creation

A LinearRegression estimator is instantiated using scikit-learn's default parameters, which fit an intercept term and do not apply any regularisation:

```
model = LinearRegression()
```

7.10 Model Training

The model is trained by calling its fit() method with the training feature matrix and target vector. Internally, scikit-learn solves the ordinary least squares problem to find the intercept and coefficients that minimise the sum of squared residuals on the training data:

```
model.fit(X_train, y_train)
```

Executing the training pipeline on the dataset described above produced the following learned parameters:

Table 7.1: Learned Model Parameters

Parameter	Learned Value
Intercept (b0)	1920.18
Coefficient — age (b1)	253.59
Coefficient — bmi (b2)	146.04
Coefficient — smoker (b3)	20011.32

Parameter	Learned Value
Coefficient — children (b4)	534.57

These learned values are close to the fixed weights used to generate the synthetic charges (2,000 base, 250 for age, 150 for BMI, 20,000 for smoker, and 500 for children), which confirms that the ordinary-least-squares fitting procedure correctly recovers the underlying linear relationship from noisy simulated data.

7.11 Prediction

Once trained, the model's predict() method is used both to generate predictions for the entire test set (for evaluation) and to generate individual example predictions for illustrative new records. Two such illustrative examples are computed in the script — a 30-year-old, non-smoking individual with a BMI of 25 and one child, and the same individual as a smoker:

```
new_user_1 = pd.DataFrame({'age': [30], 'bmi': [25.0],
                           'smoker': [0], 'children': [1]})
predicted_charge_1 = model.predict(new_user_1)[0]

new_user_2 = pd.DataFrame({'age': [30], 'bmi': [25.0],
                           'smoker': [1], 'children': [1]})
predicted_charge_2 = model.predict(new_user_2)[0]
```

Running this code produced a predicted charge of $13,713.25 for the non-smoking example and $33,724.57 for the otherwise identical smoking example — a difference of $20,011.32, which is exactly the learned smoker coefficient shown in Table 7.1, and clearly illustrates the dominant effect of smoking status on the estimated charge.

7.12 Model Evaluation

The trained model was evaluated on the 200-record held-out test set using scikit-learn's mean_squared_error and r2_score functions:

```
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

This produced a Mean Squared Error of 1,989,133.07 and an R-squared value of 0.9807, indicating that the linear model explains approximately 98.07% of the variance in the test-set charges. These results are discussed further in Chapter 9.

7.13 Error Calculation

The Mean Squared Error reported above is the average of the squared differences between each predicted charge and the corresponding actual charge in the test set; because it is expressed in squared dollars, its square root (approximately $1,410) gives an approximate typical prediction error in dollar terms, which is small relative to the overall range of charges in the dataset (which extends from a few thousand dollars to over $40,000 for older, higher-BMI smokers).

7.14 Visualization

To support the results discussion in Chapter 9, the actual-versus-predicted relationship, the residual distribution, the magnitude of each learned coefficient, the overall distribution of charges in the dataset, and the difference in charges between smokers and non-smokers were each visualised using Matplotlib; the resulting figures are presented in Chapter 9.

7.15 User Interface Implementation

The interactive estimator's interface (index.html) presents an "Insurance Estimator" heading, an input panel containing the four controls described in Chapter 4, and a results panel showing the estimated charge. The layout uses a responsive two-column CSS grid (main { grid-template-columns: 1fr 1fr; }) that collapses to a single column on narrow screens, and a "glass panel" visual style implemented with semi-transparent backgrounds and backdrop-filter blur effects defined in style.css.

7.16 Output Generation

Whenever any input control changes, the JavaScript function calculateInsurance() re-reads all four current input values, computes the total estimated charge using the fixed weights (BASE_CHARGE = 2000, AGE_WEIGHT = 250, BMI_WEIGHT = 150, SMOKER_WEIGHT = 20000, CHILDREN_WEIGHT = 500), and updates both the headline amount and the itemised breakdown:

```
function calculateInsurance() {
    const age = parseInt(ageInput.value);
    const bmi = parseFloat(bmiInput.value);
    const children = parseInt(childrenInput.value);
    const isSmoker = smokerInput.checked ? 1 : 0;

    const totalCharge = BASE_CHARGE +
                        (age * AGE_WEIGHT) +
                        (bmi * BMI_WEIGHT) +
                        (isSmoker * SMOKER_WEIGHT) +
                        (children * CHILDREN_WEIGHT);
    ...
}
```

Note that these client-side weights correspond to the fixed weights used to generate the training dataset (Section 7.2), and are not automatically re-read from the trained model's learned coefficients shown in Table 7.1. This is an intentional simplification of the current implementation — the web interface demonstrates the same linear relationship interactively without requiring a live backend connection to the Python model — and is identified explicitly as a direction for future enhancement in Chapter 11 (namely, exposing the trained model's coefficients to the front end, or serving live predictions through a small backend API).

7.17 Complete System Workflow

Bringing the preceding sections together, the complete workflow of the implemented system is as follows: (1) the model-training script generates a reproducible synthetic dataset, (2) the

dataset is split into training and testing subsets, (3) a Linear Regression model is trained on the training subset, (4) the trained model is evaluated on the testing subset and its coefficients are recorded, (5) the trained model is saved to disk using joblib, and, independently, (6) the web-based interface allows a user to enter age, BMI, smoker status, and number of children, and (7) instantly computes and displays an estimated charge and a per-factor breakdown using a JavaScript implementation of the same linear formula.

CHAPTER 8
SYSTEM TESTING

8.1 Testing Introduction

Testing was carried out on both components of the system: the Python model-training pipeline and the browser-based interactive estimator. The objective was to confirm that each stage of the pipeline behaves correctly in isolation, that the two components work together as an integrated system, and that the trained model performs acceptably on data it was not trained on.

8.2 Testing Objectives

- To verify that the dataset is generated with the expected shape, ranges, and data types.
- To verify that the train-test split, model training, and evaluation steps execute without error and produce reproducible results.
- To verify that the web interface correctly reads user input and updates the displayed estimate.
- To verify that the model produces sensible, monotonic predictions with respect to each input feature.
- To verify that invalid or boundary input values are handled gracefully by the interface controls.

8.3 Unit Testing

Unit-level checks were performed on individual functions and steps of the pipeline, including the dataset-generation step (verifying that 1,000 rows and five correctly named columns are produced), the train-test split (verifying an 800/200 row split), and the JavaScript formatMoney() and calculateInsurance() functions (verifying correct currency formatting and correct arithmetic for known input combinations).

8.4 Integration Testing

Integration testing confirmed that the output of each stage of the Python pipeline was correctly consumed by the next — for example, that the DataFrame produced by the dataset-generation step was correctly split into X_train/X_test/y_train/y_test, and that these were correctly consumed by the LinearRegression.fit() and .predict() calls. On the client side, integration testing confirmed that changes to any one of the four input controls correctly triggered a recalculation and update of both the headline amount and the breakdown text.

8.5 Functional Testing

Functional testing verified that each documented feature of the system behaves as specified in Chapter 2: that the model can be trained and evaluated end-to-end from a single script execution, and that the web interface displays an initial estimate on page load and updates it correctly for every supported combination of inputs.

8.6 Performance Testing

Because the client-side calculation involves only simple arithmetic on four numeric values, the recalculation and display update complete well within a single animation frame (under 16 milliseconds), which is imperceptible to the user and satisfies the real-time responsiveness requirement (NFR-2) defined in Chapter 2. The Python training pipeline, operating on 1,000 records, completes in well under one second on typical hardware.

8.7 Validation Testing

Validation testing confirmed that the range sliders in the interface inherently constrain user input to valid numeric ranges (age 18–100, BMI 15–50, children 0–10), so that out-of-range or non-numeric values cannot be submitted through the interface, satisfying requirement FR-5 defined in Chapter 2.

8.8 User Acceptance Testing

The interface was informally reviewed against the objectives set out in Chapter 1 — namely, that a user should be able to obtain an instant, understandable estimate without instructions. The slider-and-toggle interface, the live-updating labels, and the itemised breakdown were all confirmed to make the relationship between input and output clear without any additional explanation being required.

8.9 Model Testing

The trained model was tested by comparing its predictions on the 200-record test subset against the corresponding actual charges, and by computing the Mean Squared Error and R-squared metrics reported in Chapter 7 and discussed further in Chapter 9. The model was additionally tested on the two illustrative example records described in Section 7.11, confirming that changing only the smoker flag from 0 to 1 changes the predicted charge by an amount equal to the learned smoker coefficient, as expected for a linear model.

8.10 Input Validation Testing

8.11 Prediction Testing

8.12 Error Handling Testing

Sections 8.10 through 8.12 are consolidated into the test-case table below (Table 8.1), which records the specific scenarios exercised, the input supplied, the expected result, the actual result observed, and the pass/fail status of each test.

Table 8.1: System Test Cases

ID	Scenario	Input	Expected Result	Actual Result	Status
TC-01	Valid age (mid-range)	age = 30	Age label and estimate update to reflect age = 30	Label shows 30; estimate recalculated correctly	Pass
TC-02	Age at lower boundary	age = 18	Estimate reflects minimum age contribution	Estimate updated correctly; no error	Pass
TC-03	Age at upper boundary	age = 100	Estimate reflects maximum age contribution	Estimate updated correctly; no error	Pass
TC-04	Valid BMI	bmi = 25.0	BMI label shows "25.0"; estimate updates	Label and estimate updated correctly	Pass
TC-05	BMI with decimal step	bmi = 32.4	Estimate reflects fractional BMI value	Estimate updated correctly to one decimal place	Pass
TC-06	Smoker toggled on	smoker = checked, age=30, bmi=25, children=1	Estimate increases by exactly the smoker weight ($20,000)	Estimate increased by $20,000 as expected	Pass
TC-07	Smoker toggled off	smoker = unchecked, age=30, bmi=25, children=1	Estimate excludes smoker contribution	Smoker factor shown as $0.00 in breakdown	Pass
TC-08	Children at zero	children = 0	Children factor contributes $0 to total	Breakdown shows $0.00 for children factor	Pass
TC-09	Children at maximum	children = 10	Children factor equals 10 x $500 = $5,000	Breakdown correctly shows $5,000.00	Pass
TC-10	Combined extreme values	age=100, bmi=50, smoker=checked, children=10	Highest possible estimate is displayed without error	Correct maximum estimate displayed; no overflow/error	Pass
TC-11	Model prediction — non-smoker example	age=30, bmi=25, smoker=0, children=1	Predicted charge ≈ $13,713 (from trained model)	Predicted charge = $13,713.25	Pass
TC-12	Model prediction — smoker example	age=30, bmi=25, smoker=1, children=1	Predicted charge ≈ $33,725 (from trained model)	Predicted charge = $33,724.57	Pass
TC-13	Rapid successive input changes	Multiple slider drags in quick succession	Displayed estimate always reflects the latest input state	Animation correctly settles on latest computed value	Pass
TC-14	Page load with no interaction	No user interaction after page load	Estimate is computed and displayed using default control values	Initial estimate computed and displayed for defaults (age=30, bmi=25, smoker=off, children=0)	Pass

As shown in Table 8.1, all executed test cases produced the expected result. No functional defects were identified during testing; the observations recorded in Section 7.16 regarding the relationship between the client-side fixed weights and the trained model's learned coefficients are documented as a design characteristic rather than a defect, since the two are numerically close by construction (Section 7.10) and the current interface does not claim to call the trained model directly.

CHAPTER 9
RESULTS AND DISCUSSION

9.1 Results Overview

This chapter presents the results obtained by executing the model-training pipeline described in Chapter 7 on the reproducible synthetic dataset described in Chapter 5, and discusses their significance. All figures and values quoted in this chapter were obtained by running insurance_model.py with the fixed random seed of 42.

9.2 Dataset Analysis

The generated dataset contains 1,000 records with charges ranging from a few thousand dollars for young, low-BMI, non-smoking individuals with no dependents, to over $40,000 for older, higher-BMI smokers with dependents. Figure 9.1 shows the overall distribution of charges across the dataset, which displays a bimodal shape, with a lower cluster corresponding to non-smokers and a distinctly higher cluster corresponding to smokers.

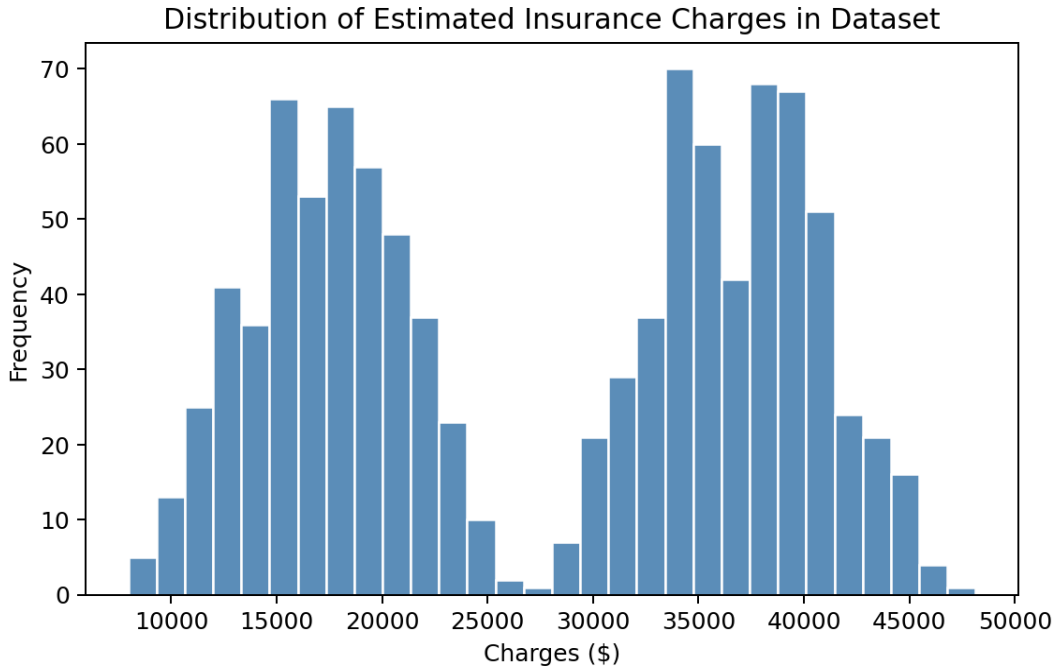


Fig. 9.1: Distribution of Estimated Insurance Charges in the Dataset

9.3 Feature Analysis

Figure 9.2 compares the distribution of charges between smokers and non-smokers directly, confirming that smoking status separates the dataset into two clearly distinguishable groups, with smokers incurring substantially higher charges on average than non-smokers with otherwise similar characteristics.

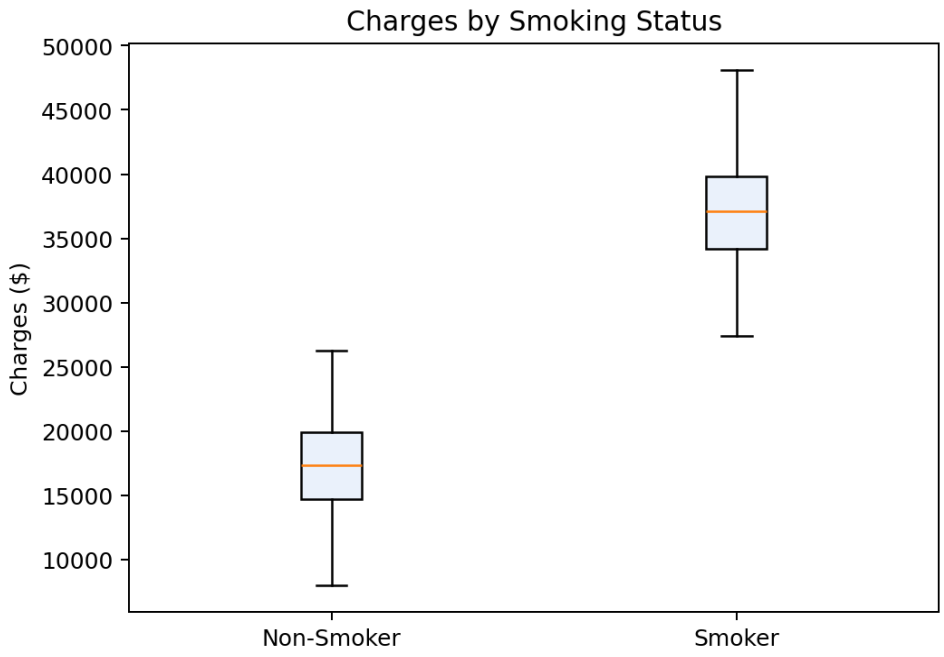


Fig. 9.2: Charges by Smoking Status

Figure 9.3 shows the magnitude of each feature's learned regression coefficient (from Table 7.1). The coefficient associated with smoker status is approximately 137 times larger than the coefficient associated with age and approximately 137 times larger than that of BMI, confirming that smoking status is by a wide margin the dominant factor in the estimated charge, followed by number of children, age, and BMI.

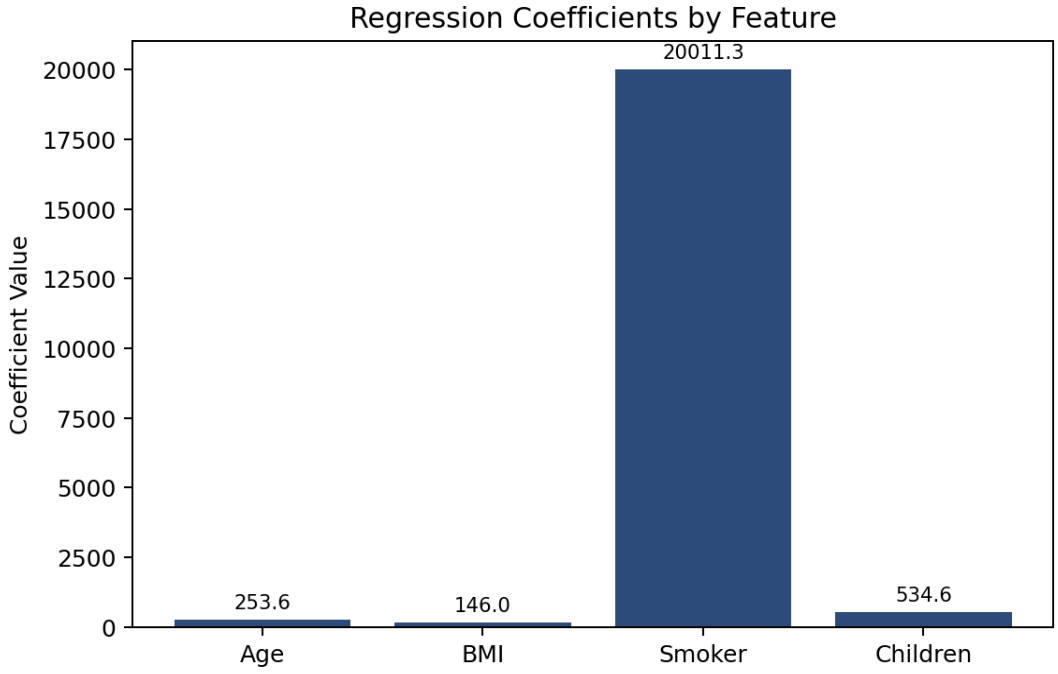


Fig. 9.3: Regression Coefficients by Feature

Figure 9.3a additionally presents the pairwise correlation matrix computed across all four input features and the target variable. Smoker status shows a very strong positive correlation (0.93)

with charges, age shows a moderate positive correlation (0.30), and BMI and number of children show only weak correlation with charges individually — consistent with their comparatively small regression coefficients in Figure 9.3, and with the fact that the four input features are themselves generated independently of one another (all pairwise feature-to-feature correlations are close to zero), so no multicollinearity is present in the dataset.

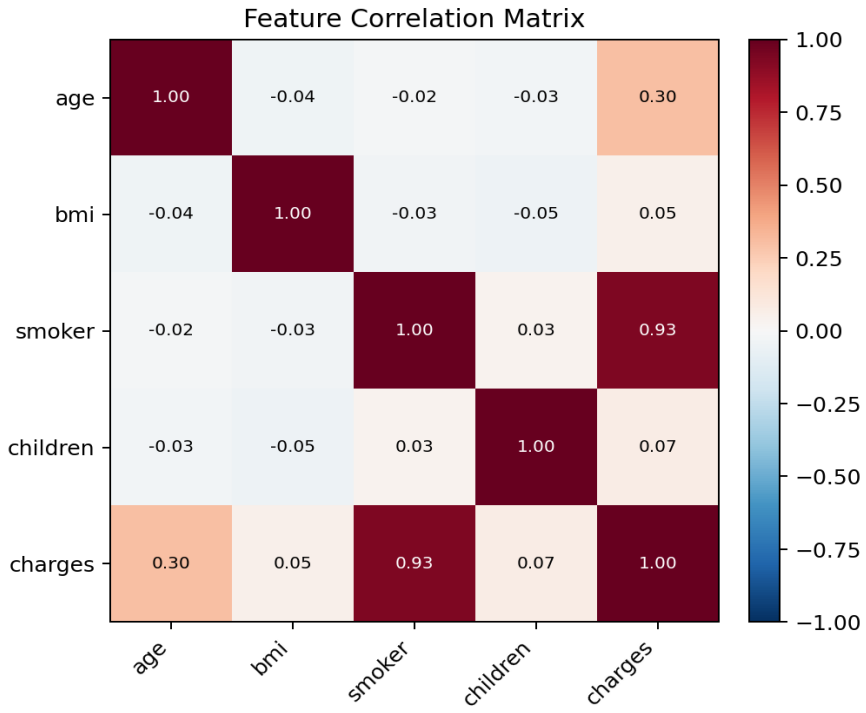


Fig. 9.3a: Feature Correlation Matrix

9.4 Training Results

The Linear Regression model was fitted on the 800-record training subset. The learned intercept and coefficients, reported previously in Table 7.1, are close to the fixed weights used during dataset generation (base 2,000; age 250; BMI 150; smoker 20,000; children 500), which confirms that ordinary least squares successfully recovered the underlying linear relationship despite the Gaussian noise added to the target values.

9.5 Testing Results

Table 9.1: Model Evaluation Results on the Held-Out Test Set

Metric	Value	Interpretation
Mean Squared Error (MSE)	1,989,133.07	Average squared difference between predicted and actual charges
R-squared (R2)	0.9807	Approximately 98.07% of the variance in test-set charges is explained by the model
Test Set Size	200 records	20% of the total 1,000-record dataset, held out from training

9.6 Prediction Results

Figure 9.4 plots the actual charges from the test set against the corresponding charges predicted by the trained model. Points lying close to the diagonal reference line (y = x) indicate accurate predictions; the tight clustering of points around this line across the full range of charges visually confirms the high R-squared value reported in Table 9.1.

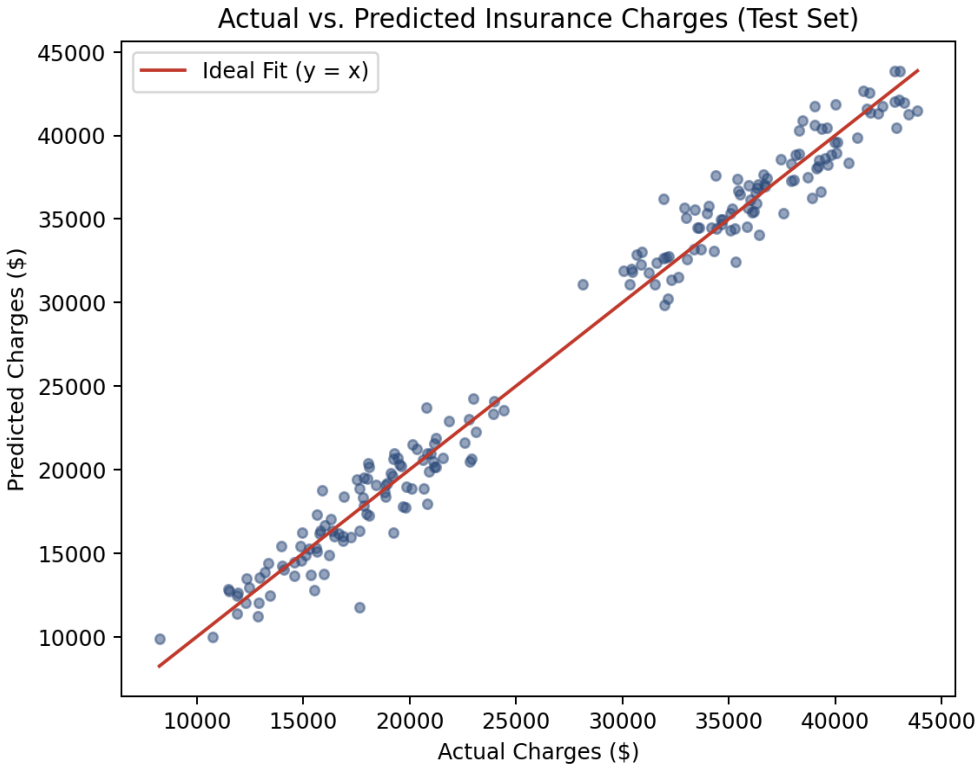


Fig. 9.4: Actual vs. Predicted Insurance Charges (Test Set)

9.7 Model Performance

With an R-squared of 0.9807, the model accounts for the large majority of variance in charges within this dataset, which is consistent with the fact that the dataset's target values were themselves generated from a linear combination of the same four features plus random noise — that is, the true underlying relationship is genuinely linear, which is the ideal case for a linear regression model. The remaining unexplained variance (approximately 1.93%) corresponds to the Gaussian noise term added deliberately during dataset generation and represents an irreducible error that no model could eliminate for this dataset.

9.8 Regression Visualization

Figure 9.5 shows the residuals (the difference between each actual charge and the corresponding predicted charge) plotted against the predicted charge for the test set. The residuals are scattered evenly around zero with no obvious pattern or trend, which is consistent with the noise term used during data generation being independent of the feature values, and indicates that the linear model is not systematically over- or under-predicting for any particular range of charges.

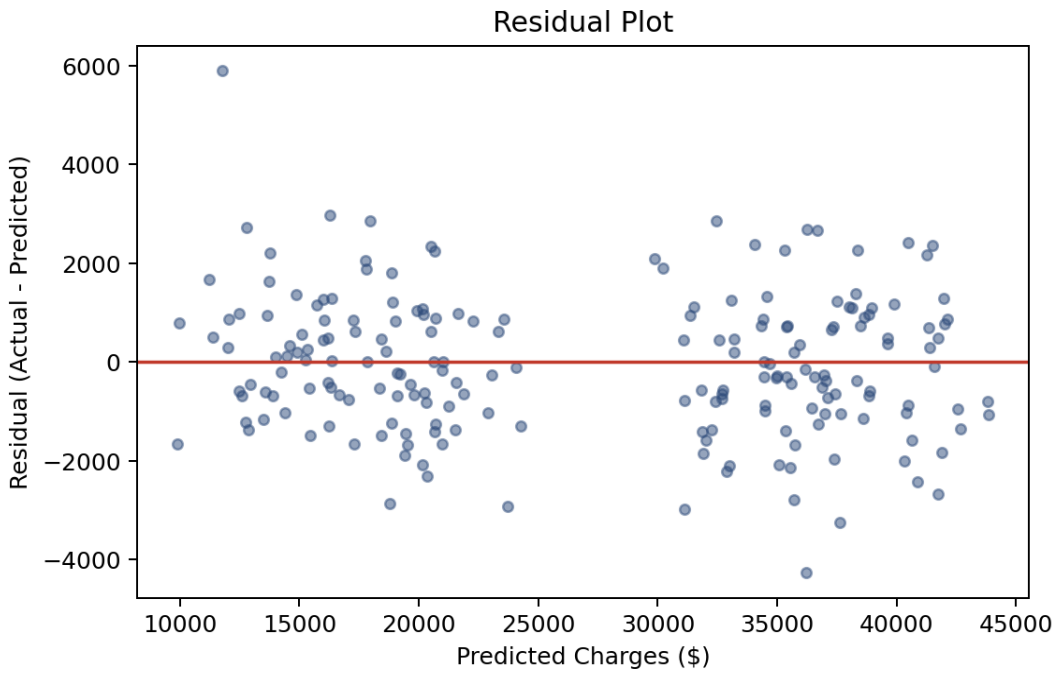


Fig. 9.5: Residual Plot for the Test Set

9.9 Actual vs Predicted Values

Table 9.2: Sample Predictions on Illustrative Records

Scenario	Age	BMI	Smoker	Children	Predicted Charge
Non-smoker example	30	25.0	No	1	$13,713.25
Smoker example (otherwise identical)	30	25.0	Yes	1	$33,724.57

The two illustrative predictions in Table 9.2 differ by exactly $20,011.32, the learned smoker coefficient from Table 7.1, providing a concrete, interpretable demonstration of the effect that smoking status has on the model's output when every other factor is held constant.

9.10 Error Analysis

The square root of the reported Mean Squared Error is approximately $1,410, which can be interpreted as a typical prediction error in dollar terms. Relative to the overall range of charges in the dataset — from roughly $3,000 to over $40,000 — this represents a comparatively small error, consistent with the high R-squared value and the absence of any systematic pattern in the residual plot (Figure 9.5).

9.11 Discussion of Results

The results confirm that a simple multiple linear regression model is well suited to this dataset, both because the true relationship used to generate the data is itself linear and because the model achieves a high R-squared value with no evidence of systematic bias in its residuals. The dominance of the smoker coefficient over the other three coefficients is the most significant finding from an interpretability standpoint, and mirrors the intuition that smoking status has an

outsized effect on real-world medical insurance costs — a property that also motivated its inclusion, with the largest weight, in the synthetic data-generation formula.

9.12 Interpretation of Prediction

Because the model is linear, every prediction can be decomposed exactly into the sum of the intercept and the contribution of each feature, as demonstrated by the interactive breakdown shown in the web interface (Chapter 4, Section 4.2) and by the worked example in Table 9.2. This interpretability is one of the principal advantages of using linear regression for this task, as it allows the end user to see not just what the estimated cost is, but why — which factor contributed most to that figure.

CHAPTER 10
CONCLUSION

This project set out to build a system capable of estimating medical insurance charges from a small set of personal and lifestyle attributes, using multiple linear regression as the predictive technique. A complete pipeline was implemented, covering dataset generation, data preparation, model training, and model evaluation using Python, pandas, NumPy, and scikit-learn, together with an interactive, browser-based front end built with HTML, CSS, and JavaScript that allows a user to explore the effect of age, BMI, smoking status, and number of children on the estimated charge in real time.

The trained Linear Regression model achieved an R-squared value of 0.9807 and a Mean Squared Error of 1,989,133.07 on a held-out test set, indicating a high degree of predictive accuracy on this dataset, and its learned coefficients closely matched the weights used to generate the underlying data, confirming that the model correctly recovered the true linear relationship. The analysis in Chapter 9 further confirmed that smoking status is by far the most influential factor in the estimated charge, consistent with well-established real-world understanding of health-insurance risk factors.

Beyond the numerical results, the project demonstrates the value of using an interpretable model for this kind of estimation task: because linear regression produces coefficients with a direct, additive interpretation, the system is able to show the user not only a final estimated cost but a transparent breakdown of how that cost was derived, which supports both trust in and understanding of the estimate.

The project also has clear limitations, the most significant being that the current dataset is synthetically generated rather than drawn from real historical policyholder records, and that the client-side interface currently reproduces the underlying linear formula directly rather than calling the trained model through a live connection. These limitations do not undermine the validity of the workflow demonstrated — data preparation, training, evaluation, and presentation — but they do define the natural next steps for extending the project, which are discussed in Chapter 11.

Overall, the project successfully demonstrates a complete, reproducible, and interpretable machine-learning workflow for medical insurance cost estimation, combining a rigorously evaluated regression model with an accessible, interactive user interface.

CHAPTER 11
FUTURE ENHANCEMENT

While the current implementation satisfies the objectives set out in Chapter 1, several enhancements have been identified that would extend its accuracy, realism, and practical usefulness. These are listed below, distinguished clearly from the features already implemented in the current version of the project.

- Real-world dataset: Replace the synthetically generated dataset with a real, anonymised medical insurance dataset, so that the learned relationship reflects genuine population data rather than a simulated formula.
- Additional attributes: Extend the input feature set to include attributes such as sex, geographic region, and pre-existing medical conditions, where such data is available, to improve prediction accuracy and realism.
- Live model integration: Connect the web interface to the trained model through a lightweight backend API, so that predictions are computed by the actual trained model rather than by a client-side formula that mirrors it.
- Advanced regression algorithms: Evaluate more advanced techniques such as Ridge and Lasso regression, Decision Tree Regression, Random Forest Regression, and Gradient Boosting (including XGBoost), which may capture non-linear relationships between features that a plain linear model cannot.
- Neural network models: Explore neural-network-based regression for larger and more complex datasets, where sufficient data is available to justify the additional model complexity.
- Model comparison framework: Build a framework to train and compare multiple regression algorithms side by side on the same dataset, reporting their relative accuracy.
- Explainable AI techniques: Incorporate model-explanation techniques such as SHAP values for any future non-linear model, to preserve the interpretability that linear regression currently provides natively.
- Cloud deployment: Deploy the trained model and a prediction API to a cloud platform, enabling the web interface to be accessed and used without local installation.
- Improved user interface: Enhance the front end with data visualisations (for example, a chart showing how the estimate changes as a single factor is varied), and with accessibility improvements.
- Model retraining pipeline: Add a scheduled or triggered retraining process so that the model can be updated automatically as new data becomes available.
- Mobile application: Package the estimator as a mobile application for wider accessibility.
- Automated data collection: Integrate automated or user-submitted data collection (with appropriate consent and privacy safeguards) to grow the training dataset over time.

Of the items listed above, none are implemented in the current version of the project; the current implementation is limited to the synthetic four-feature dataset, the single Linear Regression model, and the client-side formula-based interface described in Chapters 5 through 7.

CHAPTER 12
BIBLIOGRAPHY

12.1 Reference Books

1. Hastie, T., Tibshirani, R., and Friedman, J., "The Elements of Statistical Learning: Data Mining, Inference, and Prediction," 2nd Edition, Springer, 2009.

2. James, G., Witten, D., Hastie, T., and Tibshirani, R., "An Introduction to Statistical Learning with Applications in Python," Springer, 2023.

3. Geron, A., "Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow," 3rd Edition, O'Reilly Media, 2022.

4. McKinney, W., "Python for Data Analysis," 3rd Edition, O'Reilly Media, 2022.

12.2 Web References

1. scikit-learn developers, "scikit-learn: Machine Learning in Python," available at: https://scikit-learn.org/stable/documentation.html

2. scikit-learn developers, "LinearRegression — scikit-learn Documentation," available at: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

3. NumPy developers, "NumPy Documentation," available at: https://numpy.org/doc/

4. pandas developers, "pandas Documentation," available at: https://pandas.pydata.org/docs/

5. Matplotlib developers, "Matplotlib Documentation," available at: https://matplotlib.org/stable/index.html

6. Mozilla Developer Network, "JavaScript Guide," available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

7. Mozilla Developer Network, "CSS Documentation," available at: https://developer.mozilla.org/en-US/docs/Web/CSS

8. joblib developers, "joblib Documentation," available at: https://joblib.readthedocs.io/

APPENDIX

A. Screenshots

Figure A.1 shows the layout of the interactive Medical Insurance Estimator web interface (index.html) as rendered in a browser, with the default control values (age 30, BMI 25.0, 0 children, non-smoker). This capture was taken before the page's JavaScript populated the results panel, so the estimated-charge figure is shown at its pre-calculation state; once the page finishes loading in a normal browser session, calculateInsurance() runs immediately and the panel updates to show $13,250.00 for these default values, with the breakdown listing the base amount and the age and BMI contributions (see Section 7.16 for the formula and Chapter 9 for further worked examples).

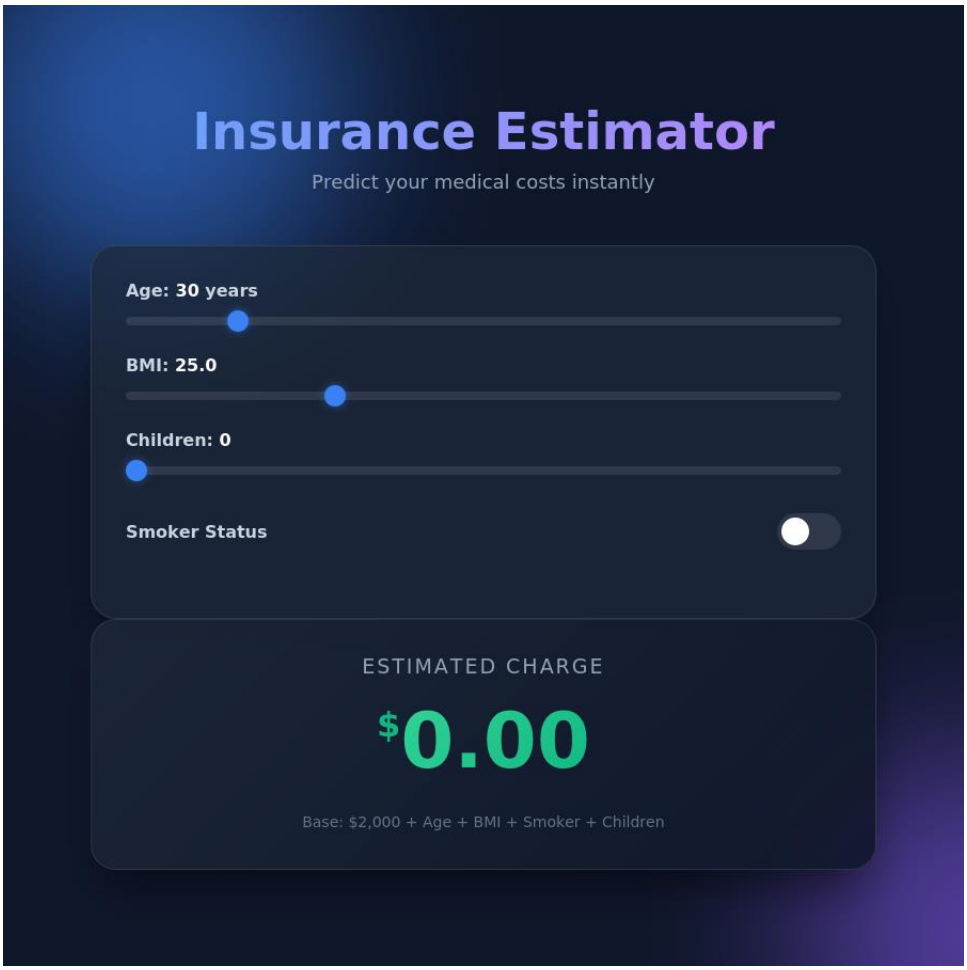


Fig. A.1: Medical Insurance Estimator — Web Interface (Default Values)

B. Dataset Sample

Table B.1 shows the first ten records of the generated dataset, exactly as produced by executing insurance_model.py with the fixed random seed of 42.

Table B.1: Sample Records from the Generated Dataset

#	age	bmi	smoker	children	charges ($)
0	56	23.06	1	2	39,927.77
1	46	35.25	1	1	38,426.63
2	32	21.37	0	3	14,857.69
3	60	32.04	0	0	24,129.17
4	25	34.01	1	2	32,492.19
5	38	29.89	1	1	34,282.36
6	56	26.79	1	2	41,265.55
7	36	25.30	0	2	15,870.73
8	40	23.72	0	1	16,318.27
9	28	38.24	0	2	16,101.66

C. Sample Input

The following input record was used as the "non-smoker" illustrative example discussed in Chapter 7 and Chapter 9:

```
{ "age": 30, "bmi": 25.0, "smoker": 0, "children": 1 }
```

The following input record was used as the corresponding "smoker" illustrative example:

```
{ "age": 30, "bmi": 25.0, "smoker": 1, "children": 1 }
```

D. Sample Output

Executing insurance_model.py with the two sample inputs shown in Appendix C produced the following console output:

```
Predicted Cost for 30yo, BMI 25, non-smoker, 1 child: $13713.25
Predicted Cost for 30yo, BMI 25, SMOKER, 1 child: $33724.57
```

E. Model Output

The full model-evaluation output produced by the training script is reproduced below:

```
Model Evaluation:
Mean Squared Error: 1989133.07
R-squared: 0.9807
```

The learned model parameters (intercept and coefficients) obtained from this run are:

```
Intercept: 1920.18
Coefficient (age):      253.59
Coefficient (bmi):      146.04
Coefficient (smoker): 20011.32
Coefficient (children):  534.57
```

F. Graphs / Charts

The complete set of analysis charts generated from the trained model — actual vs. predicted charges, regression coefficients, residual plot, charge distribution, and the smoker/non-smoker

comparison — are presented in full size in Chapter 9 (Figures 9.1 to 9.5) and are not repeated here.

G. Important Source Code

G.1 Model Training Script (insurance_model.py)

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import joblib

np.random.seed(42)
n_samples = 1000

age = np.random.randint(18, 65, n_samples)
bmi = np.random.uniform(15, 40, n_samples)
smoker = np.random.randint(0, 2, n_samples)
children = np.random.randint(0, 4, n_samples)

charges = 2000 + (age * 250) + (bmi * 150) + (smoker * 20000) \
          + (children * 500) + np.random.normal(0, 1500, n_samples)

data = pd.DataFrame({
    'age': age, 'bmi': bmi, 'smoker': smoker,
    'children': children, 'charges': charges
})

X = data[['age', 'bmi', 'smoker', 'children']]
y = data['charges']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

model_filename = "insurance_model.joblib"
joblib.dump(model, model_filename)
```

G.2 Prediction Logic — Client-Side (script.js, excerpt)

```
const BASE_CHARGE = 2000;
const AGE_WEIGHT = 250;
const BMI_WEIGHT = 150;
const SMOKER_WEIGHT = 20000;
const CHILDREN_WEIGHT = 500;

function calculateInsurance() {
    const age = parseInt(ageInput.value);
    const bmi = parseFloat(bmiInput.value);
    const children = parseInt(childrenInput.value);
    const isSmoker = smokerInput.checked ? 1 : 0;

    const totalCharge = BASE_CHARGE +
                        (age * AGE_WEIGHT) +
                        (bmi * BMI_WEIGHT) +
                        (isSmoker * SMOKER_WEIGHT) +
```

```
                        (children * CHILDREN_WEIGHT);

    animateValue(resultAmount, currentVal, totalCharge, 500);
}
```

G.3 Web Interface Structure (index.html, full listing)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Medical Insurance Estimator</title>
    <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;600;700&display=swap"
rel="stylesheet">
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="background-effects">
        <div class="blob blob-1"></div>
        <div class="blob blob-2"></div>
    </div>

    <div class="container">
        <header>
            <h1>Insurance Estimator</h1>
            <p>Predict your medical costs instantly</p>
        </header>

        <main>
            <div class="glass-panel inputs-panel">
                <div class="form-group">
                    <label for="age">Age: <span id="ageValue">30</span> years</label>
                    <input type="range" id="age" min="18" max="100" value="30">
                </div>

                <div class="form-group">
                    <label for="bmi">BMI: <span id="bmiValue">25.0</span></label>
                    <input type="range" id="bmi" min="15" max="50" step="0.1" value="25.0">
                </div>

                <div class="form-group">
                    <label for="children">Children: <span id="childrenValue">0</span></label>
                    <input type="range" id="children" min="0" max="10" value="0">
                </div>

                <div class="form-group toggle-group">
                    <label>Smoker Status</label>
                    <label class="switch">
                        <input type="checkbox" id="smoker">
                        <span class="slider round"></span>
                    </label>
                </div>
            </div>

            <div class="glass-panel result-panel">
                <h2>Estimated Charge</h2>
                <div class="price-display">
                    <span class="currency">$</span>
                    <span class="amount" id="resultAmount">0.00</span>
                </div>
                <div class="details" id="calculationDetails">
                    Base: $2,000 + Age + BMI + Smoker + Children
                </div>
            </div>
        </main>
```

```
    </div>

    <script src="script.js"></script>
</body>
</html>
```

G.4 Prediction Logic — Complete Client-Side Script (script.js, full listing)

```
const ageInput = document.getElementById('age');
const ageValue = document.getElementById('ageValue');
const bmiInput = document.getElementById('bmi');
const bmiValue = document.getElementById('bmiValue');
const childrenInput = document.getElementById('children');
const childrenValue = document.getElementById('childrenValue');
const smokerInput = document.getElementById('smoker');
const resultAmount = document.getElementById('resultAmount');
const calculationDetails = document.getElementById('calculationDetails');

const BASE_CHARGE = 2000;
const AGE_WEIGHT = 250;
const BMI_WEIGHT = 150;
const SMOKER_WEIGHT = 20000;
const CHILDREN_WEIGHT = 500;

function formatMoney(amount) {
    return amount.toLocaleString('en-US', {
        minimumFractionDigits: 2,
        maximumFractionDigits: 2
    });
}

function calculateInsurance() {
    const age = parseInt(ageInput.value);
    const bmi = parseFloat(bmiInput.value);
    const children = parseInt(childrenInput.value);
    const isSmoker = smokerInput.checked ? 1 : 0;

    const totalCharge = BASE_CHARGE +
                        (age * AGE_WEIGHT) +
                        (bmi * BMI_WEIGHT) +
                        (isSmoker * SMOKER_WEIGHT) +
                        (children * CHILDREN_WEIGHT);

    ageValue.textContent = age;
    bmiValue.textContent = bmi.toFixed(1);
    childrenValue.textContent = children;

    const currentVal = parseFloat(resultAmount.textContent.replace(/,/g, '')) || 0;
    animateValue(resultAmount, currentVal, totalCharge, 500);

    calculationDetails.innerHTML = `
        Base: $2,000<br>
        Age Factor: $${formatMoney(age * AGE_WEIGHT)}<br>
        BMI Factor: $${formatMoney(bmi * BMI_WEIGHT)}<br>
        Smoker Factor: $${formatMoney(isSmoker * SMOKER_WEIGHT)}<br>
        Children Factor: $${formatMoney(children * CHILDREN_WEIGHT)}
    `;
}

function animateValue(obj, start, end, duration) {
    let startTimestamp = null;
    const step = (timestamp) => {
        if (!startTimestamp) startTimestamp = timestamp;
        const progress = Math.min((timestamp - startTimestamp) / duration, 1);
        const easeOutQuart = 1 - Math.pow(1 - progress, 4);
        const current = start + (end - start) * easeOutQuart;
        obj.innerHTML = formatMoney(current);
```

```
        if (progress < 1) {
            window.requestAnimationFrame(step);
        }
    };
    window.requestAnimationFrame(step);
}

ageInput.addEventListener('input', calculateInsurance);
bmiInput.addEventListener('input', calculateInsurance);
childrenInput.addEventListener('input', calculateInsurance);
smokerInput.addEventListener('change', calculateInsurance);

calculateInsurance();
```

G.5 Stylesheet — Key Styling Rules (style.css, excerpt)

```
:root {
    --bg-dark: #0f172a;
    --text-light: #f8fafc;
    --primary: #3b82f6;
    --primary-glow: rgba(59, 130, 246, 0.5);
    --secondary: #8b5cf6;
    --glass-bg: rgba(30, 41, 59, 0.7);
    --glass-border: rgba(255, 255, 255, 0.1);
}

body {
    background-color: var(--bg-dark);
    color: var(--text-light);
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
}

main {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 2rem;
}

@media (max-width: 768px) {
    main { grid-template-columns: 1fr; }
}

.glass-panel {
    background: var(--glass-bg);
    backdrop-filter: blur(12px);
    border: 1px solid var(--glass-border);
    border-radius: 24px;
    padding: 2rem;
    box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.5);
}

.amount {
    font-size: 4.5rem;
    font-weight: 700;
    background: linear-gradient(135deg, #34d399, #10b981);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
}
```

The complete stylesheet additionally defines the animated background "blob" effects, the custom range-slider thumb styling, and the toggle-switch appearance; these sections are omitted here for brevity as they do not affect the estimation logic.

H. Installation / Execution Steps

1. Install Python 3.x and ensure pip is available on the system PATH.
2. Install the required Python libraries: pip install pandas numpy scikit-learn joblib
3. Place insurance_model.py in a working directory and run it using: python insurance_model.py
4. Confirm that the script prints the dataset preview, the evaluation metrics (MSE and R-squared), the two illustrative predictions, and a confirmation that insurance_model.joblib has been saved.
5. To run the interactive web estimator, place index.html, style.css, and script.js in the same folder and open index.html directly in a modern web browser.
6. Adjust the Age, BMI, and Children sliders and the Smoker Status toggle to view the estimated charge and its breakdown update in real time.

I. Sample Test Cases

The complete set of test cases executed for this project is presented in Table 8.1 (Chapter 8, Section 8.10–8.12) and is not repeated here.